Regularization in Salary Prediction A Data Scientist predicts salaries using education, experience, age, skills, and several other features. Some features are highly correlated.

```
# 1..Explain why Regularization may be useful.
Regularization reduces overfitting by adding a penalty to large model coefficients.
It helps improve model performance and handles highly correlated features.
```

```
# Compare Lasso and Ridge Regression.
Lasso (L1): Can make some coefficients exactly zero, so it performs feature selection.
Ridge (L2): Shrinks coefficients towards zero but usually does not make them exactly zero.
```

```
# Explain how Lasso can perform Feature Selection
Lasso makes the coefficients of less important features exactly zero. Therefore,
 it automatically removes unnecessary features and selects important ones.
```

```
# Describe when Elastic Net may be appropriate
Elastic Net is useful when many features are highly correlated. It combines Lasso
(L1) and Ridge (L2) penalties to perform feature selection and control overfitting.
```

Practical Python Project: Feature Selection Create a house price prediction dataset with at least 20 observations and these features:

```
# 1.Create a Pandas DataFrame.
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
data = {
    "Area": [800, 1000, 1200, 1500, 1800, 2000],
    "Bedrooms": [2, 2, 3, 3, 3, 4],
    "Age": [5, 8, 3, 10, 6, 4],
    "Bathrooms": [1, 1, 2, 2, 2, 2],
    "Random_ID": [101, 102, 103, 104, 105, 106],
    "Price": [2500000, 3000000, 3500000, 4200000, 4800000,
              5500000]
}

df=pd.DataFrame(data)
```

```
df.shape
```

```
(6, 6)
```

```
# 2..Separate the input features and target variable
x= df.drop('Price',axis=1)
y=df['Price']

print(f' my Input feature is :{x.head()}')
print(f'my target variable is :{y.head()}')
```

```
 my Input feature is :   Area  Bedrooms  Age  Bathrooms  Random_ID
0   800         2    5          1        101
1  1000         2    8          1        102
2  1200         3    3          2        103
3  1500         3   10          2        104
4  1800         3    6          2        105

my target variable is :0    2500000
1    3000000
2    3500000
3    4200000
4    4800000
Name: Price, dtype: int64
```

```
#3. Split the dataset into training and test sets.
x_train,x_test ,y_train,y_test=train_test_split(
    x, y, test_size=0.2, random_state=42
)
print(x_train)
print(x_test)
print(y_train)
print(y_test)
```

```
(4, 5)
(2, 5)
5    5500000
```

```
2    3500000
4    4800000
3    4200000
Name: Price, dtype: int64
0    2500000
1    3000000
Name: Price, dtype: int64
```

```
# 4.Apply SelectKBest with f_regression to select two features.
from sklearn.feature_selection import SelectKBest ,f_regression  # SelectKBest()    Best features select karta hai

select_best_obj=SelectKBest(score_func=f_regression,k=2)    # f_regression: Features aur target ke linear relationship ko s
                                                            # k=2   Sirf 2 best features select karta hai
x_train_selected=select_best_obj.fit_transform(x_train,y_train)
select_feature=x_train.columns[select_best_obj.get_support()]

print(select_feature)
print('\n')
print(x_train_selected)
```

```
Index(['Area', 'Bedrooms', 'Random_ID'], dtype='object')


[[2000    4  106]
 [1200    3  103]
 [1800    3  105]
 [1500    3  104]]
```

```
# 5.Display the selected feature names.
select_feature=x_train.columns[select_best_obj.get_support()]
print(select_feature.tolist())   # tolist() converts the result into a Python list.
```

```
['Area', 'Bedrooms', 'Random_ID']
```

```
# 6.Train a Linear Regression model using the selected features.
from sklearn.linear_model import LinearRegression
model=LinearRegression()
model.fit(x_train,y_train)
print(f' my model coefficient is :{model.coef_}')
print(f'my model intercept is :{model.intercept_}')
```

```
 my model coefficient is :[  1827.25445473 264543.627291     9090.90909091      0.
  88187.29994518]
my model intercept is :-8596900.849173307
```

```
# 7.Calculate the test R² score.
from sklearn.metrics import r2_score
y_pred=model.predict(x_test)
r2=r2_score(y_test,y_pred)
print(f'test R² score->{r2}')  #r2_score() actual aur predicted values compare karke R² score calculate karta hai.
```

```
test R² score->0.5724847644480326
```

```
# 8.exlain why `Random_ID` may not be useful and whether the results support that conclusion.
Random_ID is only a unique identification number assigned to each house.
It has no meaningful relationship with house prices.
It does not provide useful information for predicting house prices.
If SelectKBest does not select Random_ID, it supports the conclusion that this feature is not useful.
```

Customer Churn Prediction A telecom company has collected customer data containing the following features:

```
# 1.Explain why Feature Selection is useful for this problem.
Feature Selection removes irrelevant and unnecessary features.
It reduces the complexity of the model.
It improves model performance and may reduce overfitting.
It reduces training time.
It helps identify the most important factors affecting customer churn.
```

```
# 2. Identify features that may be irrelevant to prediction.
Customer ID: It is only a unique identification number for each customer.
Random Number: It contains randomly generated values and has no meaningful relationship with customer churn.
```

```
# 3. Explain the Filter, Wrapper, and Embedded approaches.
A. Filter Method
- Selects features using statistical techniques.
- It does not depend on a Machine Learning model.
- It is fast and simple.
```

```
- Examples: Correlation, Chi-Square, ANOVA.
B. Wrapper Method
- Selects features by training and evaluating a Machine Learning model.
- It checks different combinations of features.
- It can provide good results but is computationally expensive.
- Examples: Forward Selection, Backward Elimination, RFE.
C. Embedded Method
- Performs feature selection during model training.
- It combines the benefits of filter and wrapper methods.
- Examples: Lasso (L1 Regularization), Decision Tree Feature Importance.
```

```
# 4. Suggest two suitable Feature Selection techniques.
1. Chi-Square Test
- Used for categorical features and classification problems.
- Measures the relationship between categorical features and the target variable.
- Useful for selecting relevant categorical features such as Contract Type.
2. Random Forest Feature Importance
- Uses a Random Forest model to calculate feature importance.
- Identifies which features contribute more to predicting Churn.
- Helps rank features according to their importance.
```

```
# 5. Explain how you would prevent data leakage while selecting features.
First, split the dataset into training and testing sets.
Apply Feature Selection only on the training data.
Use fit() on training data to learn the important features.
Apply transform() on both training and testing data using the same selector.
Never use test data during feature selection or model training.
```

Start coding or generate with AI.

Start coding or generate with AI.